

LungVision AI — Full Code Walkthrough (Presentation Script)

1. The big picture

LungVision AI has two completely separate prediction paths that both end up in the same Flask app:

- **Image path:** a user uploads a CT scan → a CNN classifies it → two classical ML models (Random Forest, KNN) double-check the CNN using features it extracted from the image.
- **Numeric path:** a user fills out a 15-question symptom form → a separate Random Forest and KNN (trained on questionnaire data, not images) classify it independently.

Both paths predict the same 4 classes: `adenocarcinoma`, `large.cell.carcinoma`, `normal`, `squamous.cell.carcinoma`.

There are 4 Python files involved, and they run in a strict order:

1. `train_cnn.py` — trains the CNN, produces `cancer.h5`
2. `train_model.py` — uses the CNN to train RF + KNN on image features
3. `train_numeric_models.py` — trains a *second* RF + KNN on symptom data (independent of steps 1-2, can run anytime)
4. `app.py` — the Flask web app; loads all 5 saved model files and serves predictions live

Say this line out loud as your transition: *"Training happens once, offline, and produces model files. The Flask app never trains anything — it just loads what's already been trained and runs predictions in real time."*

2. `train_cnn.py` — training the image classifier

What problem this file solves

We have thousands of CT scan images sorted into 4 folders (one per cancer type) and we need a model that can look at a *new* image and say which of the 4 folders it would belong in.

Step-by-step

Folder structure is the labeling system. There's no spreadsheet of labels — Keras's `flow_from_directory()` walks through `train/`, `valid/`, and `test/`, and uses each subfolder's *name* as the class label. Every image inside `train/adenocarcinoma/` is automatically tagged `adenocarcinoma`. This is the simplest possible way to clean and label image data: organize-by-folder.

Normalization. `ImageDataGenerator(rescale=1./255)` divides every pixel value (which ranges 0-255) down into the 0-1 range. Neural networks train far more reliably on small, consistent numbers than on raw 0-255 integers.

Resizing. Every image — regardless of its original size — gets resized to `(150×150)` pixels (`IMG_SIZE = (150, 150)`). The CNN's input layer expects a fixed shape, so this step forces consistency across the whole dataset.

Augmentation — the actual "cleaning for robustness" step. Only the *training* generator gets these extra settings:

```
rotation_range=15, width_shift_range=0.1, height_shift_range=0.1,  
horizontal_flip=True, zoom_range=0.1
```

On every epoch, Keras generates slightly rotated, shifted, flipped, or zoomed versions of the training images on the fly. The validation and test generators don't get this — they're evaluated on the real, unmodified images. The point of augmentation is to stop the network from memorizing exact pixel positions of training images, forcing it to learn the *pattern* of each disease instead of overfitting to specific photos.

The CNN architecture. It's a `Sequential` stack of 4 convolution blocks, each one `Conv2D → BatchNormalization → MaxPooling2D`. Explain it like this in your talk: *"Each block looks for increasingly complex patterns — the first block picks up edges and textures, the deeper blocks combine those into shapes and structures."* After the 4 blocks, `Flatten()` turns the 2D feature maps into a 1D list of numbers, `Dense(256, activation='relu')` compresses that down to exactly 256 numbers — this is the most important layer in the whole project, because it's reused later — and the final `Dense(4, activation='softmax')` turns those 256 numbers into 4 probabilities that sum to 100%.

Training controls. `EarlyStopping` watches the validation loss and stops training automatically if it stops improving for 5 epochs straight (instead of blindly running all 30 epochs and risking overfitting). `ReduceLROnPlateau` shrinks the learning rate by half if progress stalls for 3 epochs, letting the model take smaller, more careful steps as it gets close to its best result.

Output: `model.save(SAVE_PATH)` writes `cancer.h5` into the `model/` folder. A side-by-side accuracy/loss plot (`training_results.png`) is also saved so you can show training curves on a slide.

3. `train_model.py` — transfer learning to Random Forest and KNN

Why this file exists

Random Forest and KNN can't sensibly look at a raw 150×150 color image — that's 67,500 individual numbers per image, far too many and far too noisy for those algorithms. But we don't want to throw away everything the CNN already learned either. The solution is **transfer learning**: reuse the CNN's already-learned visual understanding as the *input* to a different, simpler model.

Step-by-step

Load the trained CNN and cut off its head.

```
python

cnn = load_model(CNN_PATH)
extractor = tf.keras.Sequential(cnn.layers[:-1])
```

`cnn.layers[:-1]` takes every layer *except* the last one — the final `Dense(4, softmax)` layer that turns features into class probabilities. What's left, `extractor`, stops right at the 256-number `Dense(256)` layer. Feed an image in, and instead of getting back "92% adenocarcinoma," you get back 256 raw numbers — a compact numerical "fingerprint" of that image.

Extract features for every training and test image. `load_split_as_features()` walks through each class folder, loads every image, resizes it to `(150×150)`, normalizes it to 0–1 (same preprocessing as training), and runs it through `extractor.predict()`. The result: every image becomes a 256-number vector instead of 67,500 raw pixels. This is the "transfer" — the heavy lifting of understanding what's visually important in a lung CT scan was already done by the CNN; RF and KNN now train on that distilled understanding rather than starting from scratch on pixels.

Train Random Forest and KNN on those 256-number vectors. `GridSearchCV` automatically tries several combinations of hyperparameters (RF: number of trees and max depth; KNN: number of neighbors) and keeps whichever combination scores best on cross-validation. This avoids manually guessing the right settings.

Save and report. `rf_model.pkl` and `knn_model.pkl` get saved into `model/`, and `rf_knn_comparison.png` is a simple bar chart of their test accuracy side by side — good for a comparison slide.

The one-sentence summary to say in your talk: *"Instead of training Random Forest and KNN on raw pixels, we let the CNN do the seeing, and train RF and KNN on what the CNN saw — that's transfer learning."*

4. `train_numeric_models.py` — the parallel, image-free pipeline

Why it's different

There's no public dataset connecting self-reported symptoms (smoking, age, wheezing, etc.) to specific cancer subtypes — so this script generates a synthetic dataset by rule, instead of cleaning a real one.

Step-by-step

Generate 6,000 synthetic patients. Each one gets randomly assigned values for 15 features, but not pure randomness — some features are *dependent* on others to mimic real correlations. For example:

```
python

yellow_fingers = (smoking & (np.random.rand(N) > 0.4)).astype(int)
```

Yellow fingers can only be ① if the patient is also a smoker — this mirrors the real-world fact that yellow fingers are a nicotine-staining side effect, not an independent random symptom.

Decide cancer vs. normal with a weighted risk formula:

```
python

risk = 0.35*smoking + 0.15*(age>60) + 0.10*chronic_disease + ...
has_cancer = (risk > 0.45).astype(int)
```

Smoking is weighted heaviest, matching its real-world status as the leading risk factor; age over 60 is next; and so on down the list.

Decide the subtype using three competing "profile" scores — only calculated for the patients flagged as having cancer:

- `squamous_score` leans on heavy smoking + wheezing + chest pain (matches squamous cell carcinoma's real-world link to smoking and central/airway involvement)
- `adeno_score` leans on shortness of breath + younger age + non-smoking (adenocarcinoma is the subtype most commonly seen in non-smokers)
- `large_score` leans on fatigue + chronic disease + swallowing difficulty

Whichever score is highest for that patient becomes their assigned subtype. This is rule-based labeling, not learned — it exists purely so RF and KNN have *something* structured to

learn from.

Scale the features before training KNN:

```
python

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
```

This step matters specifically for KNN, because KNN classifies by measuring *distance* between data points. `Age` ranges from 30–85 while every symptom flag is just 0 or 1 — without scaling, age would dominate every distance calculation purely because its numbers are bigger, not because it's actually more medically important. Random Forest doesn't need this step, since it splits data point-by-point rather than measuring distance.

Train and save. Random Forest trains on the raw (unscaled) features, KNN trains on the scaled version, and both — plus the scaler itself, since new patients need to be scaled the same way at prediction time — get saved as `numeric_rf_model.pkl`, `numeric_knn_model.pkl`, `numeric_scaler.pkl`.

5. `app.py` — the Flask app that ties everything together

Lazy loading

Every model — `cancer.h5`, `rf_model.pkl`, `knn_model.pkl`, `numeric_rf_model.pkl`, `numeric_knn_model.pkl`, `numeric_scaler.pkl` — has its own `get_*_model()` function that only loads the file the *first* time it's needed, then keeps it cached in a global variable for every request after that:

```
python

def get_model():
    global _model
    if _model is None:
        _model = load_model(CNN_PATH)
    return _model
```

This means the app starts up fast, and only pays the (slow) cost of loading a large model the first time someone actually uses that feature.

`build_result_block()` — the shared formatter

Both the image path and the numeric path funnel their raw predictions through this one function:

```
python
```

```
def build_result_block(class_idx, probs):
```

Given a predicted class index and a list of 4 probabilities, it looks up the human-readable label, the color category (`cancer` vs `normal`), the list of educational facts about that class, and bundles it all into one dictionary. This is why the "Model Comparison & Agreement" card looks identical whether it's comparing CNN/RF/KNN on an image or RF/KNN on a questionnaire — they all produce the exact same shaped data structure.

Route: `/predict` (the image path)

1. User uploads a file → saved into `upload/` with a unique filename.
2. The CNN loads the image, resizes to `150×150`, normalizes, and predicts → gives the *primary* result.
3. `extract_cnn_features()` runs the *same* image through the truncated feature-extractor (just like in `train_model.py`) to get its 256-number fingerprint.
4. That fingerprint is fed into both `rf_model.predict_proba()` and `knn_model.predict_proba()` to get the secondary opinions.
5. All three results, plus two matplotlib plots (RF feature importance, KNN nearest-neighbor distances) get passed to `predict.html` to render.

Route: `/predict_numeric` (the numeric path)

1. User submits the symptom form via JavaScript `fetch()` (not a normal page reload — `predict_numeric.html`'s `submitForm()` collects the form into `FormData` and POSTs it).
2. The 15 form values get converted into a single ordered numeric array, matching the exact column order used during training (`NUMERIC_FEATURE_ORDER`) — this order *must* match, or the model would silently misinterpret which number means what.
3. `numeric_rf_model.predict_proba()` runs directly on that array.
4. The scaler transforms the array, then `numeric_knn_model.predict_proba()` runs on the scaled version.
5. Both results, plus their own two plots (symptom importance, nearest neighbor distances), get passed to `predict_numeric.html`.

Why two different prediction flows feel like one product

Both routes end with the exact same pattern: primary model → secondary model → `build_result_block()` → comparison card showing agreement or disagreement. The only difference is *what* goes in at the very start — a photo, or a form.

6. The full request, end to end (good closing slide)

Walk through this sequence live if you want a strong finish:

1. User logs in → `session['logged_in'] = True` is stored in the Flask session (a server-side cookie-backed dictionary).
2. User clicks **Predict (Numeric)** → browser navigates to `/predict_numeric` (GET) → Flask checks the session, renders the empty form.
3. User fills the form, clicks **Analyze Risk** → JavaScript intercepts the click, builds a `FormData`, and `fetch()`-POSTs it to the same `/predict_numeric` URL.
4. Flask's `predict_numeric()` function now sees `request.method == 'POST'`, pulls the form values, runs both models, and renders the same template — but this time with `rf_prediction` and `knn_prediction` filled in, so the result cards appear.
5. The JavaScript receives that full rendered HTML page back as text and replaces the entire document with it (`document.write`), which is why the page appears to "reload" with results even though it never left `/predict_numeric`.

One sentence to end on: *"Every prediction in this app — whether it started as a photo or a form — goes through the same shape: extract features, run two models, format the result the same way, and let the user see where the models agree."*